

ECLIPSE SDV HACKATHON 2026 - HACK TO THE FUTURE – SIX-IN-SYNC

1. Your Team at a Glance

Identity and roster

Six engineers. One portable safety loop. We bring together autonomous-driving, embedded-systems, functional-safety, and hardware expertise to build a demonstrable vehicle-safety prototype.

Team member	Background	GitHub handle
Lindsay Shantha Rubia K	M.Eng. Autonomous Driving student, Hochschule Coburg	lindsayk09
Till Winter	Bachelor's engineering student, Embedded Systems; dual study at Duale Hochschule Baden-Württemberg / Mercedes-Benz	ProjectfirstLearner
Sören Fox	Bachelor's engineering student, Embedded Systems; dual study at Duale Hochschule Baden-Württemberg / ZF	SoerenFox
Venkata Krishnaiah Vemula	Functional Safety Architect, AUMOVIO	vkrishnaiahv
Matthias Musch	High-Speed Interface Engineer, SiliconAuto	mm-sa
Anoop Ramananda	Senior Hardware Engineer, SiliconAuto	anoop1702

Responsibilities

Four members currently focus on technical exploration and implementation; two focus on requirements, organization, documentation, and coordination. We will assign a named owner to each GitHub task. Responsibilities remain flexible, and individual technical roles will follow expertise and immediate project needs.

Challenge alignment and core idea

We are competing in “Hack to the Future.” Guardian Loop will monitor child presence and cabin temperature in a parked-vehicle scenario, evaluate risk, and request a warning or protective action. Our key objective is to replace simulated components with real hardware without rewriting the Guardian decision logic.

We will begin with simulated occupancy and temperature. Sensor events will reach Guardian through uProtocol publish/subscribe. Guardian will request window opening through uProtocol RPC; an actuation adapter will translate the request to SOVD REST, and OpenSOVD CDA will communicate with the controller through UDS over DoIP. The OpenBSW-based controller will execute the action and report its result.

AutoSD will host Guardian on the supplied vehicle computer. openDuT will manage test connections and support switching between simulated and physical endpoints. Hardware-specific logic will remain in adapters and controllers, behind stable service interfaces.

Priorities and demonstration

Essential: A working virtual sensing–decision–actuation loop with visible risk states and action results.

Full challenge target: The same Guardian artifact running on AutoSD, an openDuT-managed configuration switch, and at least one real endpoint replacing a simulated one.

Optional: Fan or alarm control, mock notifications, and themed display integration only after the core demonstration works.

2. How Do You Work?

Development Process

We use a lightweight Kanban-based development process to plan and track the project throughout the three-day hackathon. All tasks are managed in a shared GitHub Project, providing the team with a single overview of responsibilities, priorities, and progress. Tasks are organized by status, priority, technical area, and hackathon day, with the workflow progressing from Backlog and Todo to In Progress, Review/Test, and finally Done.

The three-day schedule is divided into clear iterations: Day 1 – Foundation, focusing on the development environment, core Guardian logic, and basic sensor simulation; Day 2 – Integration, focusing on communication and the integration of sensors, Guardian, and actuation; and Day 3 – Demo & Stabilization, focusing on end-to-end testing, debugging, documentation, and preparing the final demonstration.

Tasks are assigned to team members and kept small enough to allow progress to be tracked continuously. This lightweight structure gives the team a clear overview of what needs to be done while remaining flexible enough to react quickly to technical challenges and changing priorities during the hackathon.

Quality Control

To ensure a consistent level of quality throughout the project, we will use a lightweight but structured quality process. Core functionality, especially the Guardian state machine, will be covered by unit tests to verify the expected behavior for different child-presence and temperature scenarios. In addition, integration and end-to-end tests will be used to validate the communication between the simulated sensors, Guardian logic, and actuation layer.

All code changes will be developed in separate branches and reviewed through pull requests before being merged into the main branch. This provides a second pair of eyes for important changes and helps identify bugs or design issues early. We will also keep the project documentation up to date, including the README and relevant technical decisions, so that the implementation and architecture remain understandable for the entire team.

A task is considered complete only when the implementation works as intended, relevant tests have been executed successfully, and the corresponding code has been reviewed. This approach allows us to maintain code quality and traceability while keeping the development process efficient and suitable for the limited three-day hackathon timeframe.

Team Communication

To ensure effective communication under the time constraints of the three-day hackathon, we will use a simple and direct communication structure. The shared GitHub Project serves as the central source of

truth for tasks, responsibilities, priorities, and progress, while a dedicated team chat will be used for quick day-to-day communication.

We will have short team syncs to align on current progress, upcoming tasks, and potential blockers. Team members are encouraged to communicate blockers as early as possible rather than spending excessive time on isolated problems. For complex technical issues, we will use short, focused discussions with the relevant team members to reach a solution quickly.

To avoid communication overhead, decisions that affect the overall architecture or multiple work areas will be discussed with the team, while implementation decisions within an assigned task can be made directly by the responsible developer. This structure allows us to stay aligned, resolve issues quickly, and keep the majority of our time focused on implementation and integration.

Decision Making

To handle internal bottlenecks and design disagreements efficiently, we will follow a pragmatic and time-conscious decision-making process. Technical issues will first be discussed among the team members directly involved in the affected area, with the goal of reaching a solution based on technical feasibility, project requirements, and the remaining time.

For minor implementation decisions, the responsible developer can make the decision independently to avoid unnecessary delays. Decisions that affect the overall architecture or multiple components will be discussed briefly with the team.

If no consensus can be reached within a reasonable time, the designated technical lead will make the final decision based on the project goals and available implementation time.

Blockers will be raised early through the team communication channels and the GitHub Project so that they can be addressed before they impact other work packages. Important architectural decisions and their rationale will be documented briefly to ensure transparency and avoid revisiting the same discussion later. This approach allows us to resolve disagreements constructively while keeping the project focused on delivering a functional solution within the three-day timeframe.

Prototype boundary: Use simulated occupancy, configurable demonstration thresholds, and mock emergency notifications. Hardware operation will follow the organizers' approved setup; the prototype is not a certified safety system.